

Dev Log — 1 August 2026

Session Goal

Get the build green again and get Snake actually playable. The previous log (28 July) ended with the build broken across three confirmed blockers (`UE-0052` / `0053` / `0054`) and a fourth latent crash (`UE-0055`). This session covers a single commit, `93f6bd7` ("back to green build, fixed up scenes, and starting fixing snake movement"), which bundles the entire arc: fixing every compile and link error from the previous session's unfinished refactor, chasing down a long chain of runtime crashes and rendering/movement bugs one at a time via live `gdb` inspection (screenshots were unreliable in this sandbox — it runs XWayland, and `x11grab` -style capture came back solid black even when the underlying render data was confirmed correct), and finally landing on a working, grid-locked, input-responsive snake. All code was written by hand by the user per `CLAUDE.md` 's read-only policy; Claude's role throughout was direct diagnosis (reading source, building, and running the actual binary under `gdb` rather than trusting build or commit messages) and design discussion.

The session ended by stepping back from bug-fixing entirely to design the next piece of real engine capability — a generic, attachable `MovementComponent` — rather than continuing to patch movement logic onto `SnakeSegment` / `SnakeHead` directly. That design is written up separately as `design-docs/2026-08-01_195220_movement-component-architecture.md` and is not yet implemented; it drives most of this log's **Next Session** section.

Work Completed

The three build blockers from the previous log are fixed

`Scene::Scene()` (`UE-0052`) now takes a real `SceneParams : SceneBaseParams` and initializes its `entityManager` / `gameSystemManager` / `eventBus` references from it — paired with a matching engine-side change: `Core::Scenes::SceneFactory` was changed from `std::function<unique_ptr<IScene>()>` to `std::function<unique_ptr<IScene>(const SceneBaseParams&)>`, and `SceneManager::ResolveScene` now builds a `SceneBaseParams` from its own three member references and passes it to every registered factory (`scene-manager.cpp`). This means the engine supplies those three dependencies to *every* scene automatically — `GameplaySceneParams` just inherits `SceneParams` for the free fields and adds `gameSettings` / `screenWidth` / `screenHeight` of its own, and `SnakeApplication::Initialize()` 's registration lambda only ever names game-specific data, never engine plumbing.

`EntityManager::OnRegistration()` (`UE-0053`) is implemented. `AppleSpawnSystem`, `SnakeSpawnSystem`, and `SnakeGrowSystem` (`UE-0054`) were migrated off directly touching `this->entityManager` / `this->eventBus` onto `GameSystem` 's public `AddEntity` / `RemoveEntity` / `GetEventBus()` wrappers — those members are `private` on `GameSystem` on purpose, and the three concrete systems were the last holdouts still reaching past that boundary.

Application's system registration was rebuilt around ownership, not return-type casting

`Application::RegisterSystem` returns the type-erased `Core::Systems::ISystem*` — casting that back down to `IGameSystemManager*` / `IEntityManager*` isn't safe. `Application::Initialize()` now constructs each system as a named `unique_ptr<Concrete>`, captures the concrete raw pointer *before* moving it into `RegisterSystem`, and uses that captured pointer for everything downstream (wiring `SceneManager`'s constructor args, assigning `Application::sceneManager`, which changed from an owning `shared_ptr` to a non-owning `Core::Scenes::ISceneManager*` since `SystemManager` is the real owner now). This also fixed a bug where `Application::sceneManager` was never assigned at all — `GetSceneManager()` would have returned a null reference the first time anything called it.

CMakeLists.txt's explicit source list was out of sync with the repo

Nine `.cpp` files that already existed on disk from the previous session's refactor were simply never added to the `add_executable(...)` list: `engine/scenes/scene.cpp`, `engine/systems/system.cpp`, `engine/systems/system-manager.cpp`, `engine/systems/game-systems/game-system.cpp`, `engine/systems/game-systems/game-system-manager.cpp`, `snake-game/game-components/snake-body-component.cpp`, and all three game systems (`apple-spawn-system.cpp`, `snake-growth-system.cpp`, `snake-spawn-system.cpp`). Once added, a second wave of link errors surfaced (see below), followed by a small wave of compile errors — `game-system.cpp` needed `#include "core/entities/i-entity.hpp"` (its `unique_ptr<IEntity>` destructor needs the complete type, previously only forward-declared), and `SnakeBodyComponent : Core::Components::IComponent` (`snake-body-component.hpp`) was missing `public` on its base — private inheritance by default in a `class`, which made `IEntity::GetComponent<SnakeBodyComponent>()`'s internal `static_cast` fail to see the `IComponent` base.

A cluster of "declared but never defined" link errors

Several classes declared a destructor or override in their header and never defined it in the `.cpp`: `AppleSpawnSystem`, `SnakeSpawnSystem`, and `SnakeGrowSystem`'s destructors, plus `Engine::Scenes::Scene::OnSceneExit()` and `SnakeGame::GameplayScene::Render()`. Since GCC picks a class's first non-inline virtual as its "key function" and only emits the vtable in the translation unit that defines it, an undefined destructor doesn't just leave a dangling symbol — it means the whole vtable never gets emitted, which is why these surfaced as `undefined reference to vtable for X` rather than pointing straight at the missing destructor. `SystemManager` / `GameSystemManager`'s own constructors, and `GameSystemManager::OnDebugUpdate` / `OnDebugRender` / `OnRegistration`, had the same problem from the previous session's refactor never being finished.

A dangling reference to a temporary killed entity construction with "pure virtual method called"

`EntityParams` / `SnakeSegmentParams` store `transform` as a **reference member** (`const Core::Spatial::ITransform2D&`). `SnakeSpawnSystem::CreateHead()` built a `Transform2D` temporary directly inside `SnakeHeadParams{...}` on one statement, then used the resulting `headParams` on the *next* statement — binding a temporary to a reference *member* doesn't extend its lifetime the way binding to a

reference *variable* does, so the temporary was destroyed at the end of the first statement, and `Entity`'s constructor (reading `params.transform.GetPosition()` etc.) was calling virtual methods through an already-destroyed object. Confirmed by `gdb` backtrace landing in `__cxa_pure_virtual` inside `Entity::Entity`. `AppleSpawnSystem` never hit this because its equivalent construction happens as a single expression. Fixed by giving `CreateHead()` a named local `Transform2D` that outlives both statements.

A chain of runtime bugs between "it links" and "it's playable"

Diagnosed live via `gdb` (breakpoints, `dprintf`-style logging commands, and — once — a `-DCMAKE_BUILD_TYPE=Debug` reconfigure of the gitignored `build/` dir to get real symbols), in the order they were hit:

- `SnakeSpawnSystem::head` (`SnakeHead*`) had no initializer — reading it in `!this->head` was undefined behavior, and on this run happened to look non-null, routing `Spawn()` into `Reset()` / `TeardownBody()` on a garbage pointer instead of `CreateHead()`.
- After fixing that: `SnakeBodyComponent` was read via `GetComponent<SnakeBodyComponent>()` in three places but never *added* to any entity — `SnakeHead::OnRegistration()` only added `InputComponent`. `Append()` / `GetSegments()` / `Clear()` were all being called through a null `this`.
- Entities were being spawned at `Size2D{1, 1}` (hardcoded, both in `AppleSpawnSystem` and `SnakeSpawnSystem::CreateHead`) instead of `settings.boxSize` (which already existed, default `20`, and was simply never referenced) — invisible on a 500×500 window.
- `SnakeSegment::SetDirection` called itself recursively with no base case (`this->SetDirection(this->direction)`) — unbounded recursion, stack overflow, crash on any keypress that set a non-zero direction. Compounded by `size` defaulting to `0`, which made the recursive branch's guard condition true on the very first call.
- `SnakeSegment` / `SnakeHead` never initialized `size` / `speed` / `direction` away from their bare defaults (the old, now-dead `Snake::Initialize()` used to do this) — the snake never moved on its own, and `SnakeHead::Update()` didn't even call `Move()`.
- `SnakeSpawnSystem::CreateBody()`'s per-segment position math had two bugs: the Y expression was `(y / height) * height` — an identity, so every segment landed on the *same* Y as the head — and the X expression multiplied an already-correct per-segment offset by `width` a second time, placing segments far off-screen. Net effect: one visible head, the rest of the body invisible off-canvas, reading as "just a single box."
- `SnakeHead::Update` called `this->Step()` directly, bypassing the timer-gated dispatch in `SnakeSegment::Update` entirely — the head moved a full grid cell on literally every frame once any direction was set, reading as "constantly moving."
- `SnakeSegment::Step()`'s actual position update was nested *inside* the "is there a queued direction" `if` block, instead of running unconditionally every tick — meant movement only happened on the same tick a new direction was queued.
- `SnakeSegment::BeginPlay()` seeded `direction = this->transform->GetPosition()` — a leftover from an earlier iteration, before grid-ticking existed. Confirmed live via `dprintf`-logged `Step()` calls showing every entity's `direction` exactly equal to its own spawn coordinates. Removed; `direction`

now starts at the header default (a small/ zero vector) and is only ever set to a proper unit vector via input.

- `moveInterval` was being derived as `size / speed` inside `BeginPlay()`, collapsing to a fixed 2 seconds regardless of tuning — far too slow. Settled on removing that derivation entirely and using a flat header default (`0.2f`, five steps/sec) instead of deriving it from unrelated fields.
- Movement input used `BindAxis("MoveHorizontal"/"MoveVertical", ...)`, which fires again on key-release with a neutral value — that neutral event was stomping `pendingDirection` back to `{0, 0}` before the (2-second, at the time) tick ever got to read it, making input feel almost entirely unresponsive. Replaced with four discrete, edge-triggered `Bind("Up"/"Down"/"Left"/"Right", ...)` actions (`ActionRouter::Bind`, driven by `InputActionPressedEvent`, which only fires on press) — no neutral event exists for a digital action, so there's nothing to stomp the queued direction.
- Up/Down were inverted relative to raylib's Y-down screen convention (`Up` was setting `direction.y = +1`). Swapped the two literal signs in `SnakeHead::BeginPlay()`'s `Bind` callbacks — isolated, one-line-each fix, nothing rendering- or physics-side needed to change.

Housekeeping

The old inline comment `// RAID BOSS: The Scene manager needs to be refactored to fit the ISystem spec` is gone from `scene-manager.cpp` — `SceneManager` already inherits `Engine::Systems::System` now, so that concern is resolved as a side effect of this session's other work, not something that needed its own pass.

Lessons Learned

Components can't tick in this engine today — and that's a real gap, not just a Snake problem

Checked directly: `Core::Components::IComponent` (`core/components/i-component.hpp`) is just a virtual destructor. Nothing generically calls into a component per-frame — `Render` and `Input` each solved this with their own one-off mechanism instead (`RenderComponent2DManager` walks its own registered list and calls `Render()`; `ActionRouter::Update()` polls bound axes), not a shared capability. This came up while discussing why `SnakeHead` / `SnakeSegment` had to carry their own movement state directly — there was no other way for that state to get ticked.

GCC's "key function" rule turns a missing destructor definition into a missing vtable

A declared-but-undefined virtual method — usually the destructor, since it's often the first non-inline virtual a class has — means the compiler never emits the vtable at all in any translation unit, not just a normal undefined-symbol error at the one call site that needed it. Worth remembering the next time a linker error says `undefined reference to vtable for X` instead of naming a specific missing function.

Binding a reference *member* to a temporary does not extend its lifetime

Only binding a reference *variable* to a temporary extends that temporary to the reference's own scope. A reference member initialized from a temporary argument at construction time is not covered by that rule — the temporary dies at the end of the full expression that constructed the *containing object*, even though the reference member keeps pointing at where it used to be. This is exactly what made

`SnakeSpawnSystem::CreateHead()` crash only when the `Transform2D` temporary and the `SnakeHeadParams` construction were split across two statements, and not when (as in `AppleSpawnSystem`) they happened in one.

The event bus already supports arbitrary event data — no plumbing changes needed

Checked `IEventBus::Subscribe<TEvent> / Publish<TEvent>` (`i-event-bus.hpp:19-37`): both are fully generic over the concrete event type, and `Subscribe` already round-trips the full `TEvent&` to the caller's typed callback. `GameStartedEvent / AppleCollectedEvent` are empty structs today purely because nobody's put fields on one yet, not because of a bus limitation. Directly relevant to the `MovementSteppedEvent` planned for next session.

Grid-locked vs. smooth movement, and axis vs. discrete input, for a classic Snake

Discussed whether Snake should use continuous (`speed * deltaTime`) movement or the traditional fixed-grid-cell stepping — landed on grid-locked, since it's both the genre convention and what the codebase's `accumulatedDistance / size` scaffolding and grid-snapped input math were already halfway built for. Separately, discussed continuous `Axis1D` actions (reusable across gameplay and a hypothetical settings menu) versus discrete, edge-triggered per-direction `Bind` actions — chose discrete for now, since `BindAxis` firing again on release (with a neutral value) is exactly what was breaking responsiveness, and discrete actions have no neutral event to stomp state with at all. Axis-based reuse across menu/gameplay was noted as a reasonable idea to revisit if/when a settings page actually needs shared bindings, as long as every consumer remembers to filter the neutral value itself.

Architectural Decisions

`SceneManager` assembles and injects `SceneBaseParams` at scene-resolution time

Rather than have every game-specific `*SceneParams` struct manually thread `entityManager / gameSystemManager / eventBus` through from the composition root, `SceneFactory` was changed to take `const SceneBaseParams&`, and `SceneManager::ResolveScene` builds that struct from its own members and passes it to whichever factory is registered. Game code (`SnakeApplication`, `GameplaySceneParams`) never names engine plumbing at all — it only ever supplies game-specific data, and inherits the base params struct for the rest.

Application::RegisterSystem callers capture the concrete pointer before the move, rather than casting the return value

RegisterSystem returns the type-erased **ISystem***, which can't be safely cast back to a derived interface. Instead of adding a templated wrapper or an unsafe cast, call sites in **Application::Initialize()** now construct each system as a named **unique_ptr<Concrete>**, grab the raw concrete pointer before handing ownership to **RegisterSystem**, and use that pointer for everything that needs the real type. Keeps **RegisterSystem** itself simple and avoids ownership ambiguity — **Application** never holds the **unique_ptr** itself, it's a transient local before ownership moves to **SystemManager**.

Grid-locked movement over continuous, and discrete per-direction actions over shared axes

Covered above under Lessons Learned — recording here since both were deliberate choices with a rejected alternative, not just observations. Grid-locked: chosen over continuous **speed * deltaTime** movement to match genre convention and the existing tick-based scaffolding. Discrete **Bind** actions: chosen over continuous **Axis1D** specifically because axis release events were breaking input responsiveness, and there's no equivalent failure mode for a discrete, edge-triggered action.

Movement is becoming a generic, attachable **MovementComponent**, not a **GameSystem**

The biggest decision of the session, made *before* any of it was implemented — full rationale, code sketches, and diagrams are in [design-docs/2026-08-01_195220_movement-component-architecture.md](#). Summary: a **GameSystem** implies "one engine-owned instance"; movement is a per-character trait, so it belongs on a component that can be attached to anything (the design's litmus test: attaching it to **Apple** should make Apple grid-step and take input with zero apple-specific code). This requires giving components a real **OnUpdate** hook — currently a gap in the engine, not just a Snake-specific workaround — and keeping the snake-specific "body follows head" logic entirely out of the generic component, reacting instead to a new data-carrying **MovementSteppedEvent** published on the existing (and, per Lessons Learned, already fully capable) event bus. A "should events also carry callable behavior" idea came up and was deliberately deferred as a separate, parallel concept (a request/command system) rather than folded into broadcast events — no concrete need for it yet.

Known Issues Carried Forward

- **UE-0055** — **Snake** ([game-entities/snake.hpp](#) / [.cpp](#)) and **GameplayScene::snake** (**unique_ptr<Snake>**) are still present as dead code. This session removed the call site that dereferenced it (**GameplayScene::Update()** no longer calls **this->snake->Update()**), which fixed the guaranteed crash, but the class itself — still holding a **this->length** assignment on a member that doesn't exist, a **GetCenter()** with no return statement, and fully commented-out **CheckIfShouldGrow()** / **Teleport()** — was not deleted. Recommend actually deciding and acting on this next session rather than carrying it a third time.

- **UE-0056** — 1UP: self-noted in `apple-spawn-system.cpp` this session — "Add a Subscribe function to `Engine::Scenes::Scene / GameSystem` and remove `GetEventBus()` ." A convenience wrapper so callers don't need `this->GetEventBus().Subscribe<T>(...)` and can just call `this->Subscribe<T>(...)` directly. Not blocking anything.
- **UE-0004** — BUG: Window-side event bus stub. Unchanged.
- **UE-0014** — REAPER: `GameplayState` still owns stale snake/apple references and polls input directly (`gameplay-state.cpp`), including a hand-rolled collision check against `this->snake->head` that predates the new `GameSystem` -based architecture entirely. Unchanged — not investigated this session, and its relationship to whether `GameplayState` is even reachable at runtime (see `UE-0055`) is still an open question.
- **UE-0015** — REAPER: `ManagerInterfaceManager` update/debug methods commented out. Unchanged.
- **UE-0016** — REAPER: `SnakeGameSettings::GetBoxSize()` commented out. Unchanged.
- **UE-0019** — SIDE QUEST: `BoundaryWrapSystem` . Unchanged — still fully commented out in the dead `Snake::Teleport()` ; likely moot once `UE-0055` is actually resolved.
- **UE-0020/UE-0023** — REAPER: Remove stale `stateMachine` `shared_ptr` from `Application` members. Not investigated this session.
- **UE-0021** — 1UP: `std::is_abstract_v` discriminator breaks for `ColliderComponent2D` . Not investigated this session.
- **UE-0024** — REAPER: UI component system commented out. Not investigated this session.
- **UE-0026** — REAPER: Remove `Apple` verification hooks before merge. Not investigated this session.
- **UE-0028** — MAIN QUEST: `GameplayStateMachine` use-after-move. Unchanged, still entangled with `UE-0035` .
- **UE-0029** — RAID BOSS: Unify entity/component ownership to remove the leak and double free. Unchanged in scope this session.
- **UE-0030** — 1UP: `TransformComponent2D::GetTransform()` returns a shared function-local `static` . Not touched this session.
- **UE-0031** — REAPER: Dependency injector correctness bugs. Not touched this session.
- **UE-0032** — REAPER: `ComponentDispatcher` `const_cast` ing. Not touched this session.
- **UE-0033** — REAPER: Miscellaneous cleanups from code review. Not touched this session.
- **UE-0036** — Event's `action` field never populated. Unconfirmed a third session running; recommend resolving the flag rather than carrying it again.

- **UE-0037** — Keymap array literal order didn't match `Action` enum order. Same recommendation as `UE-0036`.
- **UE-0050** — SIDE QUEST: Wire `showClientDebugLogs` into the logger properly. Not touched this session.
- **UE-0051** — RAID BOSS: Debug System needs a full refactor. Not touched this session.
- **UE-0062** — Deferred hypothetical, explicitly parked rather than pursued: could events carry callable behavior (not just data), for a request/response-style pattern running alongside the existing broadcast-style events? No concrete use case yet — revisit only if one shows up. Kept deliberately separate from `MovementSteppedEvent`'s data-only design.

Next Session

Primary goal: implement `design-docs/2026-08-01_195220_movement-component-architecture.md` in order — each step is a prerequisite for the next.

- **UE-0057** — MAIN QUEST: Add `virtual void OnUpdate(float) {}` to `IComponent` (non-pure, matching the existing `Entity / System` no-op-override convention) and have `Entity::Update()` iterate its own `components` map, calling `OnUpdate(deltaTime)` on each. Foundational — nothing else in the design doc can be built until components can tick. Deliberately not extending components to the rest of the entity lifecycle (`BeginPlay / OnActivate / OnRegistration`) yet — no need for it identified.
- **UE-0058** — MAIN QUEST: Build `MovementComponent` — owns `direction`, `pendingDirection`, `moveTimer`, `moveInterval`, and its own input binding (the four `Bind("Up"/"Down"/"Left"/"Right", ...)` calls currently on `SnakeHead`). Implements the timer-gated `Step()` (reversal-guarded direction commit, then move by `direction * cellSize`), and publishes a new `MovementSteppedEvent{ IEntity* entity; Vector2D oldPosition; Vector2D newPosition; }` each time it actually steps. Depends on `UE-0057`.
- **UE-0059** — MAIN QUEST: Strip `direction / pendingDirection / moveTimer / moveInterval / Step() / SetDirection() / input-binding` out of `SnakeSegment / SnakeHead` entirely — they go back to being plain transform/collider/render holders, like `Apple`. Attach `MovementComponent` in `SnakeHead::OnRegistration()` instead. Depends on `UE-0058`.
- **UE-0060** — MAIN QUEST: "Part 2" — `SnakeBodyComponent` subscribes to `MovementSteppedEvent`, filters to `event.entity == this->owner`, and on a match walks its segment deque **tail-to-head**, each segment taking the position the one ahead of it held *before* this tick (finishing with the frontmost segment taking the head's `oldPosition`). This is the actual "body follows head" feature. Depends on `UE-0059`.
- **UE-0061** — SIDE QUEST: Attach `MovementComponent` to `Apple` as a smoke test — confirms the component is genuinely reusable and not secretly snake-shaped, per the design doc's optional adoption step 7.

- **UE-0055** — Actually decide `Snake` 's fate this time: finish deleting it (and `GameplayScene::snake`) or commit to finishing its migration. Given `UE-0057` – `UE-0060` are about to make `SnakeSegment` / `SnakeHead` fully self-sufficient via components, deletion looks like the obvious answer, but flagging for explicit confirmation rather than assuming.
- **UE-0038** — MAIN QUEST: Migrate `Confirm` / `Decline` off the dead `InputSystem::IsActionPressed` API. Unchanged this session.
- **UE-0035** — MAIN QUEST: `GameContext::input` is never assigned; reconcile the `MainMenuState` workaround. Unchanged, still blocked on `UE-0038` (plus `UE-0014` , `UE-0028`).